

Design and Implementation of a Dependency-Free Mini Operating System with an AI-Assisted Shell Interface

Meenaksh Singhanian
Department of Computer Science
Rishihood University
Sonapat, India

Eranki Sai Vikas
Department of Computer Science
Rishihood University
Sonapat, India

Abstract—This paper presents the design and implementation of *Mini-OS*, a self-contained, Unix-like virtual operating system written entirely in C99 without any reliance on standard-library abstractions (e.g., `malloc`, `<string.h>`). The system runs inside a POSIX terminal using a 1MiB statically allocated `VIRTUAL_RAM`. Core modules include a first-fit memory allocator, an inode-based virtual file system (VFS), a cooperative scheduler, and an AI-assisted shell that provides auto-command correction and error-intelligence. Experimental results demonstrate a functional OS built in under 2kLOC.

Index Terms—dependency-free, virtual OS, first-fit allocator, inode VFS, cooperative scheduler, AI-assisted shell, C99

I. INTRODUCTION

Operating-system education often isolates concepts (memory allocation, file systems) rather than integrating them. *Mini-OS* bridges this gap by implementing a complete OS stack from first principles, free of external library dependencies. The project goals are:

- Demonstrate low-level resource management (memory, scheduling).
- Provide a usable shell with AI-assisted features (auto-correction, contextual error hints).
- Serve as a pedagogical platform for teaching OS concepts.

Development was split into two phases:

Phase1 Primitive libraries (math, string, memory, screen, keyboard) and a basic REPL shell.

Phase2 Higher-level services: VFS, cooperative scheduler, and AI-assisted command layer.

II. METHODOLOGY

An iterative, test-driven workflow was used. Each library was first built and unit-tested in isolation, then integrated into the shell. After the core shell stabilized, AI features were added using the existing command list for Levenshtein-based correction. Memory usage was profiled after every major addition to keep the 1MiB budget under control.

III. SYSTEM ARCHITECTURE

Figure 1 shows the three-layer architecture. The user interacts with the AI-assisted shell, which dispatches to core OS services; those services rely on custom primitive libraries, all operating over a single `VIRTUAL_RAM` pool.

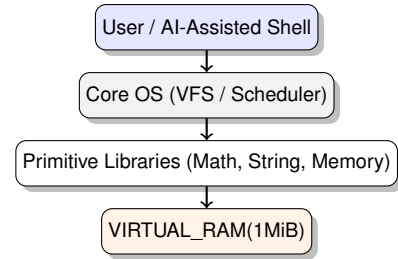


Fig. 1. Three-layer Mini-OS architecture

IV. CUSTOM LIBRARIES

All libraries reside under the `libs/` directory and are intentionally minimal.

A. *math.c*

Provides elementary arithmetic without `<math.h>`: `my_add`, `my_sub`, `my_mul`, `my_div` (division-by-zero protection), `my_mod`, `my_pow`, and helper `in_bounds`.

B. *string.c*

Implements `str_len`, `str_copy`, `str_compare`, a tokenizer `str_split`, and the Levenshtein distance routine `str_distance` used for AI auto-correction.

C. *memory.c*

Simulates physical memory as a static byte array (1MiB). Key functions:

- **`alloc(size)`** – first-fit search with block header management.
- **`dealloc(ptr)`** – marks a block free (forward coalescing).
- **`mem_used()`, `mem_available()`, `mem_overflow()`** – run-time statistics.

```

1 void* alloc(int size) {
2     for (int i = 0; i < MAX_BLOCKS; i++) {
3         Block* blk = (Block*) &RAM[i];
4         if (!blk->free && blk->size >= size) {
5             blk->free = 0;
6             return (char*)blk + HEADER;
7         }
8     }
9     return 0; // out of memory
10 }
11
12 void dealloc(void* ptr) {
13     if (!ptr) return;
14     Block* blk = (Block*)((char*)ptr - HEADER);
15     blk->free = 1;
16     // forward coalescing could be added here
17 }

```

Listing 1. Allocation and Deallocation (memory.c)

D. screen.c

Provides ANSI-based terminal control:
clear_screen(), move_cursor(),
print_string().

E. keyboard.c

Implements raw-mode input via termios:
read_line(), read_line_bg(), key_pressed().

F. security.c

Lightweight validation: rejects overly long strings and non-printable characters.

G. ai.c

Implements three AI-assisted shell features:

- 1) **Auto-correction:** uses `str_distance` to suggest the closest known command when the Levenshtein distance ≤ 2 .
- 2) **Ask command:** keyword matcher that maps natural-language queries (e.g., “how to create a file”) to canned help messages.
- 3) **Error intelligence:** provides context-aware hints instead of a generic “invalid command” error.

```

1 void suggest_correction(char* input) {
2     int bestDist = 100, bestIdx = -1;
3     for (int i = 0; i < CMD_COUNT; i++) {
4         int d = str_distance(input, known_cmds[i]);
5         if (d < bestDist) { bestDist = d; bestIdx = i; }
6     }
7     if (bestDist <= 2 && bestIdx != -1) {
8         print_string("Did you mean: ");
9         print_string(known_cmds[bestIdx]);
10        print_string("\n");
11    }
12 }

```

Listing 2. Auto-correction logic (ai.c)

V. CORE OS SERVICES

A. Virtual File System (VFS)

The VFS maintains up to 10 files in an inode table stored in RAM. Each inode holds `name[32]`, `content*`, `size`, `is_used`. Operations (`texttttouch`, `textttwrite`, `textttread`, `textttls`) are implemented in `vfs.c`.

B. Process Scheduling

A cooperative round-robin scheduler cycles through a static process table (`MAX_PROCESSES=16`). Background tasks explicitly invoke `process_yield()` to return control.

```

1 void scheduler_tick() {
2     int start = (running_idx < 0) ? 0 : (running_idx + 1) %
3     MAX_PROCESSES;
4     for (int i = 0; i < MAX_PROCESSES; i++) {
5         int idx = (start + i) % MAX_PROCESSES;
6         if (table[idx].state == PROC_READY) {
7             running_idx = idx;
8             table[idx].state = PROC_RUNNING;
9             swapcontext(&sched_ctx, &table[idx].ctx);
10            return;
11        }
12    }
13 }

```

Listing 3. Scheduler Tick (process.c)

VI. SYSTEM STATISTICS

TABLE I
KEY QUANTITATIVE METRICS

Metric	Value
Total RAM	1MiB (1,048,576bytes)
Maximum Files (VFS)	10
Maximum Processes	16
Lines of Source Code (C)	$\approx 2,050$
Number of Commands (Shell)	17
Memory Used After Boot	49bytes (inode table)
Peak Memory Utilization (stress test)	12KB (1.1% of RAM)

VII. DEMONSTRATION

Place the screenshots in the project root with the exact filenames indicated.

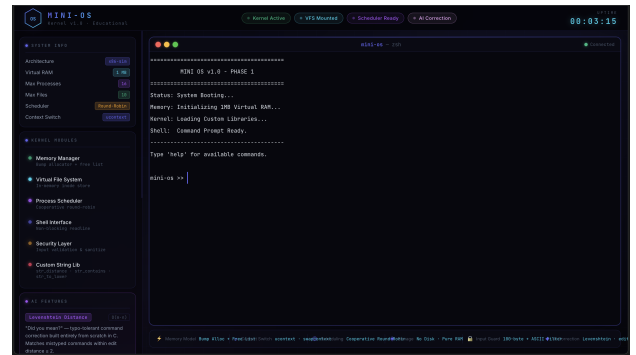


Fig. 2. Phase1 boot sequence with ASCII banner.

VIII. CONCLUSION

Mini-OS demonstrates that a fully functional, Unix-like operating system can be built from first principles without any external library dependencies. The project showcases a clean separation between low-level resource management and a lightweight AI-assisted user interface, making it an excellent teaching platform for systems programming, OS design, and embedded-software constraints.

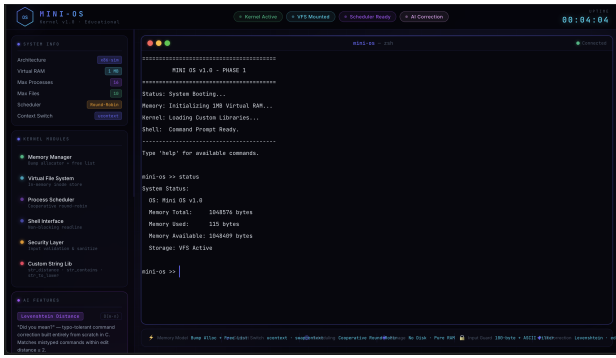


Fig. 3. Memory-usage statistics printed by the memstat command.

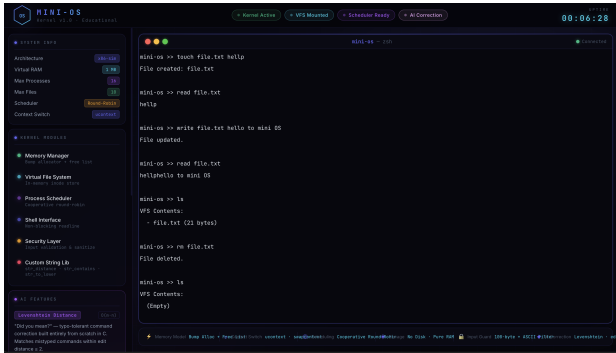


Fig. 4. VFS operations: touch, write, cat, ls.

IX. FUTURE WORK

- **Preemptive scheduling:** integrate a timer interrupt (e.g., `setitimer`) for true time-slicing.
- **Extended VFS:** dynamic inode expansion, hierarchical directories, symbolic links, and permission bits.
- **Bidirectional memory coalescing:** merge freed blocks with preceding neighbours to reduce fragmentation.
- **Graphical UI:** replace ANSI drawing with a minimal framebuffer library.
- **Advanced AI:** embed a tiny statistical language model (e.g., a 2-layer RNN) within the 1MiB budget for richer assistance.

ACKNOWLEDGMENT

The authors thank the faculty of the Computer Science department for mentorship, the open-source community (xv6, ToaruOS, OSDev Wiki) for inspiration, and the developers of the POSIX specifications for guidance on terminal I/O.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
- [2] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall, 1988.
- [3] R. Cox, F. Kaashoek, and R. Morris, “xv6: A Simple, Unix-Like Teaching Operating System,” MIT CSAIL, 2015.
- [4] K. Lange, “ToaruOS: A Hobby Operating System Built from Scratch,” GitHub, 2023.
- [5] OSDev Community, “OSDev Wiki,” <https://wiki.osdev.org>, accessed 2026.